# P3236R1
# Please reject P2786 and adopt P1144

Published Proposal, 2024-05-14

**Authors:**

Alan de Freitas
Daniel Liam Anderson
Giuseppe D'Angelo
Hans Goudey
Jacques Lucke
Krystian Stasiowski
Stéphane Janel
Thiago Maciera

**Audience:**

SG17

**Project:**

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

---

## Abstract

We library maintainers ask that WG21 reject P2786 trivial relocatability; it is unfit for our purposes. We support P1144 trivial relocatability, which is fit for our purposes.

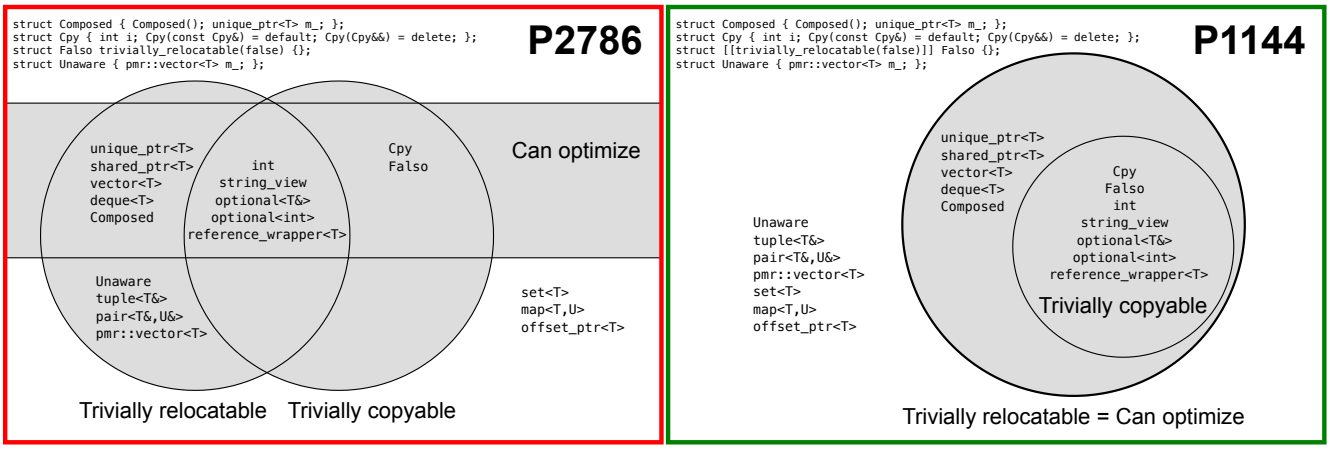
## Table of Contents

## § 1. Why not P2786?

In P2786, types with user-provided `operator=` are reported as trivially relocatable by default. It's incorrect to target these types for memmove optimization in `vector::erase`, `rotate`, and `swap`, since they don't behave value-semantically. But P2786 gives no way to distinguish these types like `tuple<int&>` from ordinary trivially relocatable types like `unique_ptr<int>`. We cannot use P2786's type-trait.

| | P2786 | P1144 |
|---|---|---|
| **Trait baseline** | `trivially_move_constructible` && `trivially_destructible` | `trivially_copyable` |
| **Libraries using that baseline** | Nobody | Abseil, AMC, BSL, Folly, HPX, Parlay, Qt |

We all use `is_trivially_copyable` as our baseline: we assume that every trivially copyable type can be trivially relocated. In P2786, the programmer is allowed to create a type that is trivially copyable but not trivially relocatable. We prefer P1144's simpler model, in which "trivially copyable types" is a proper subset of "trivially relocatable types."




In P2786, it's ill-formed to create a trivially relocatable type with an `offset_ptr` member, or even a Boost `static_vector` member. P1144 considers these use-cases so important that they form 40% of its "Design goals" section. P2786 doesn't offer a migration path for existing libraries at all.

In P2786, there is no backward-compatible syntax for marking a type trivially relocatable. P2786 proposes a new keyword `trivially_relocatable`, and goes out of its way to make the keyword unusable in C++23-and-earlier. P1144 simply uses an attribute, similar to `[[noreturn]]`, which lets us enable our optimizations in all language modes. Our libraries are widely used pre-C++26. We look forward to using P1144's marking syntax, but we cannot use P2786's.

**P2786 is deliberately incomplete.** This makes it hard to reason about. **It is a chess problem with multiple potential "lines" of development.** In contrast, P1144 is a complete proposal, with an optimizing implementation publicly available since 2018. P1144 in its current state is acceptable to us.

P2786's authors have drafted followup papers including P2959R0 and P2967R0, all with incomplete wording, attempting to fix some of its problems. P2959R0 proposes "heroic" changes to the semantics of library containers, for example `std::container_replace_with_assignment` to customize the behavior of `vector::erase`. We don't want new semantics for containers. We want safety and reliability for the optimizations we are already doing, with the semantics we already have.

## § 2. Why P1144?

P1144 provides useful semantics for `is_trivially_relocatable`.
Every signatory to this paper desires P1144 semantics (not P2786 semantics) in our libraries.
P1144, by providing stronger guarantees of value semantics, permits more optimizations:

| | P2786 | P1144 | Libraries that optimize, demanding P1144 semantics |
|---|---|---|---|
| **vector reserve** | Optimizable | Optimizable | (N/A) |
| **vector insert** | Can't be optimized | Optimizable | AMC, BSL, Folly, Qt |
| **vector erase** | Can't be optimized | Optimizable | Abseil, AMC, BSL, Folly, Qt |
| **rotate** | Can't be optimized | Optimizable | Qt |
| **swap** | Can't be optimized | Optimizable | Abseil |

P1144 provides a stable library API that some signatories have already implemented. It is well-designed and consistent with the rest of the STL. P2786 proposes only one library function — a constrained `std::trivially_relocate` — with no implementation experience. As algorithm authors, we do not want P2786's library function; it is not useful to us.

| | P2786 | Codebases providing P2786's API | P1144 | Codebases providing P1144's API |
|---|---|---|---|---|
| **Trait** | `is_trivially_relocatable` with P2786 semantics | None | `is_trivially_relocatable` with P1144 semantics | Abseil, AMC, HPX, Parlay |
| **Bulk** | constrained `trivially_relocate` | None | unconstrained `uninitialized_relocate` | AMC, Blender, EASTL, Iros, HPX, Parlay, Qt |

P2786's warrant marking can be used only if all bases and members are themselves trivially relocatable; this makes it "viral downward." As container authors, we think P2786's normalization of a large number of explicit markings will cause programmer fatigue and lead to bugs. We support P1144's approach, which requires explicit markings only in the places that the library programmer directly exposes to the client, where invariants are clearest and audits are simplest. Compare how the two proposals handle the following `optional`-like type:

| P2786 | P1144 |
|---|---|
| Three markings, two on implementation details | Just one marking, at the most easily audited level |

| Implementation details like `OptlPiece` that cannot trivially relocate in isolation, must "lie" | Implementation details do not have to "lie" |
|---|---|
| ```template<class T><br>struct OptlDetail trivially_relocatable {<br>  // ...<br>  ~OptlDetail();<br>};<br><br>struct OptlPiece trivially_relocatable {<br>  // ...<br>  OptlPiece(OptlPiece&&);<br>};<br><br>template<class T><br>struct Optional trivially_relocatable\<br>(is_trivially_relocatable_v<T>)<br>    : private OptlDetail<T> {<br>  OptlPiece m_;<br>  Optional(Optional&&);<br>  ~Optional();<br>};``` | ```template<class T><br>struct OptlDetail {<br>  // ...<br>  ~OptlDetail();<br>};<br><br>struct OptlPiece {<br>  // ...<br>  OptlPiece(OptlPiece&&);<br>};<br><br>template<class T><br>struct [[trivially_relocatable\<br>(is_trivially_relocatable_v<T>)]]<br>    Optional : private OptlDetail<T> {<br>  OptlPiece m_;<br>  Optional(Optional&&);<br>  ~Optional();<br>};``` |
| Unfit for purpose | Fit for purpose |

## § 3. Signatories

We ask that WG21 reject [P2786] and adopt [P1144] instead.

- **Amadeus AMC:** Stéphane Janel <stephane.janel@amadeus.com>
- **Blender:** Hans Goudey <hans@blender.org>
- **Blender:** Jacques Lucke <jacques@blender.org>
- **Boost:** Alan de Freitas <alandefreitas@gmail.com>
- **Boost:** Krystian Stasiowski <sdkrystian@gmail.com>
- **Carnegie Mellon Parlay:** Daniel Liam Anderson <dlanders@andrew.cmu.edu>
- **Folly:** Giuseppe Ottaviano <ott@meta.com>
- **Qt Project:** Giuseppe D'Angelo <giuseppe.dangelo@kdab.com>
- **Qt Project:** Thiago Maciera <thiago@macieira.org>
- **Ste||ar HPX:** Hartmut Kaiser <hkaiser@cct.lsu.edu>
- **Ste||ar HPX:** Isidoros Tsaousis <tsa.isidoros@gmail.com>
- **Ste||ar HPX:** Shreyas Atre <shreyasatre16@gmail.com>

# § References

## § Informative References


**[P1144]**

Arthur O'Dwyer. *std::is_trivially_relocatable*. February 2024. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p1144r10.html

**[P2786]**

Mungo Gill; Alisdair Meredith. *Trivial relocatability for C++26*. February 2024. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2786r4.pdf